

Número: _____ Nome: _____

1. Especificação (3 valores)

A função `lastSeen` calcula a última posição de um array onde um dado elemento ocorre. No caso do elemento não existir no array, a função retorna (-1).

```
int lastSeen (int x, int v[], int N){  
    // Pre: N >= 0  
    int i= N-1;  
    while (i>=0 && v[i] != x)  
        // Inv:  
        i=i-1;  
    // Pos: ???  
    return i;  
}
```

1. Complete a especificação da função de forma a refletir a descrição acima (bem como a implementação apresentada).
2. Defina um invariante apropriado para provar a correcção parcial desta função.

2. Complexidade de algoritmos recursivos (3 valores)

A função `diffConseqR` calcula o comprimento do maior sub-array sem elementos repetidos. Para isso usa a função `lastSeen` que calcula a última posição de um array onde um dado elemento ocorre (no caso do elemento não existir no array, a função retorna `-1`).

```
int diffConseqR (int v[], int N){
    int p;
    if (N<2) return N;
    p = lastSeen (v[N-1],v,N-1);
    return max(1+diffConseqR (v+p+1, N-p-2),
               diffConseqR(v,N-1));
}
```

Escreva e resolva uma recorrência que traduza o número de acessos ao array `v` no caso em que o array não tem elementos repetidos (e por isso as invocações da função `lastSeen` retornam sempre `-1`).

Número: _____ Nome: _____

3. Complexidade de algoritmos iterativos (3 valores)

A função `diffConseq` calcula o comprimento do maior sub-array sem elementos repetidos.

Com o objectivo de analisar a complexidade desta função em termos de número de acessos ao array,

1. Identifique o melhor e pior casos da execução desta função.
2. Para o pior caso identificado, calcule o número de acessos ao array.

```
int diffConseq (int v[], int N){  
    int r=0,c=1,i;  
    for (i=1; i<N; i++){  
        c = min (1+c, i-lastSeen(v[i], v, i));  
        r = max (r, c);  
    }  
    return r;  
}
```

4. Min-Heap (3 valores)

Assumindo que existem funções

- **void** bubbleUp (**int** h[], **int** i) que faz o bubble-up do elemento da posição i
- **void** bubbleDown (**int** h[], **int** N, **int** i) que faz o bubble-down do elemento da posição i

Defina a função **void** heapify (**int** v[], **int** N) que transforma o array v numa *min-heap*.

Qual a complexidade da função apresentada.

Número: _____ Nome: _____

5. Estruturas de Dados (5 valores)

MinHeap Considere que se usa um array dinâmico para armazenar uma *min-heap*

```
typedef struct dynArr {  
    int size, used;  
    int *values;  
} DynArr;
```

No qual os campos `size` e `used` correspondem ao tamanho do array `values` e ao tamanho da *min-heap*, respectivamente. Considere ainda a *min-heap* com `size=12`, `used=10` e em que os primeiros 10 elementos do array `values` têm os seguintes valores:

0	1	2	3	4	5	6	7	8	9	10	11
10	20	50	30	60	70	100	40	80	90	...	...

1. Considere que se acrescenta um novo (i.e. diferente de todos os que lá estão) elemento x a esta *min-heap*, em que $(0 < x < 100)$. Qual o número **médio** de *swaps* que serão feitos.

Resposta:

2. De forma a fazer análise amortizada do custo (escritas no array) da inserção, definiu-se como função de potencial $\phi(x) = 2 \cdot x.\text{used} - x.\text{size}$. Qual o custo amortizado de, na tabela apresentada, acrescentar o valor 25 (considere que cada swap tem um custo de 2).

Resposta:

Tabelas de Hash Considere uma tabela de hash de inteiros ($hsize=10$, $hash(x) = x\%10$), implementada em open-address com linear probing. Considere ainda que a tabela tem o seguinte conteúdo

0	1	2	3	4	5	6	7	8	9
U 20	F 74	U 42	U 23	D 14	U 52	U 36	F 17	F 12	U 19

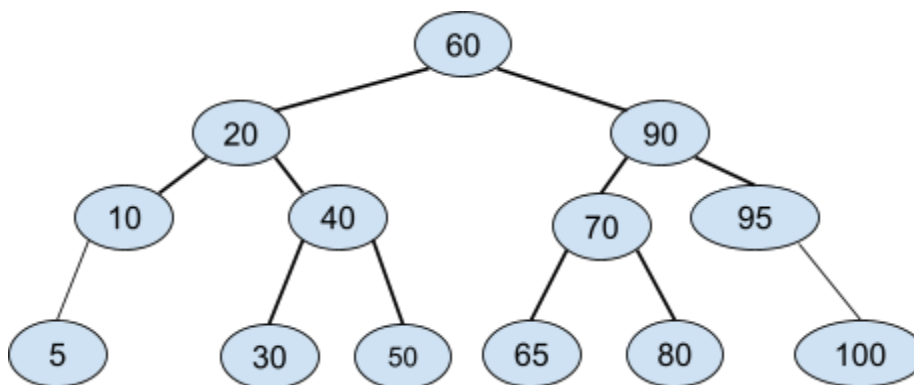
1. A inserção do valor 35 será feita em que posição?

Resposta:

2. A consulta do valor 74 quantas posições da tabela consulta?

Resposta:

Árvores AVL Considere a árvore balanceada abaixo.



Após a inserção balanceada de 45 nesta árvore, qual o menor valor dos nodos do nível 2 (assuma que a raiz se encontra no nível 1)

Resposta:

Grafos Seja g um grafo não orientado, não pesado e ligado com 15 arestas. A invocação `breadth_first(g, 0, vis, pais)` preenche o array `pais` com os seguintes valores

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
pais =	-1	5	6	8	0	6	7	11	4	6	6	13	13	8	13

Qual a distância (número mínimo de arestas) do caminho entre os vértices 0 e 7?

Resposta:

Número: _____ Nome: _____

6. Grafos (3 valores)

Considere o tipo habitual para representar árvores binárias

```
typedef struct nodo {int valor; struct nodo *esq, *dir;} *ABin;
```

Adapte o algoritmo de travessia *breadth-first* de forma a definir a função

int porNivel (**ABin** a, **int** v[], **int** N) que preenche o array v com os elementos da árvore a, por níveis. A função deverá preencher um máximo de N elementos e retornar o número de elementos preenchidos.